

Received 22 September 2023, accepted 10 October 2023, date of publication 13 October 2023, date of current version 18 October 2023.

Digital Object Identifier 10.1109/ACCESS.2023.3324458

RESEARCH ARTICLE

Offline Reinforcement Learning for Automated Stock Trading

NAMYEONG LEE AND JUN MOON^{ID}, (Senior Member, IEEE)

Department of Artificial Intelligence, Hanyang University, Seoul 04763, South Korea

Corresponding author: Jun Moon (junmoon@hanyang.ac.kr)

This work was supported in part by the National Research Foundation of Korea (NRF) Grant funded by the Ministry of Science and ICT, South Korea, under Grant NRF-2021R1A2C2094350; and in part by the Institute of Information & Communications Technology Planning and Evaluation (IITP) Grant funded by the Korea Government (MSIT) under Grant 2020-0-01373.

ABSTRACT Recently, with the increasing interest in investments in financial stock markets, several methods have been proposed to automatically trade stocks and/or predict future stock prices using machine learning techniques, such as reinforcement learning (RL), LSTM, and transformers. Among them, RL has been applied to manage portfolio assets with a sequence of optimal actions. The most important factor in investing in stocks is the utilization of past stock price data. However, existing RL algorithms applied to stock markets do not consider past stock data when taking optimal actions, as RL is formulated based on the Markov property. In other words, it means that the existing RL algorithm infers action based on the current state only. To resolve this limitation, we propose Transformer Actor-Critic with Regularization (TACR) using decision transformer to train the model with the correlation of past MDP (Markov Decision Process) elements using an attention network. In addition, a critic network is added to improve the performance by updating the parameters based on the evaluation of an action. For an efficient learning method, we train our model using an offline RL algorithm through suboptimal trajectories. To prevent overestimating the value of actions and reduce learning time, we train TACR through a regularization technique with an added behavior cloning. The experimental results using various stock market datasets show that TACR performs better than other state-of-the-art methods in terms of the Sharpe ratio and profit.

INDEX TERMS Portfolio allocation, reinforcement learning, sequence modeling, Sharpe ratio, transformer, stock market.

I. INTRODUCTION

Recently, various machine learning methods such as reinforcement learning (RL), long short-term memory (LSTM), and transformers have been studied for investments in financial stock markets. The main purpose of such studies is to predict future stock prices and/or automatically trade stocks based on specified optimization criteria. The reader is referred to [1], [2], [3], [4], [5], [6], [7], [8], and [9] and the references therein for various learning algorithms used to study financial markets. In fact, since stock prices exhibit patterns, such algorithms show remarkable performance. Specifically, the RL methods [1], [2] show high returns

and stability compared with the other algorithms [10], [11]. In addition, the transformer and LSTM [3], [6] show overall high accuracy of stock movement predictions by considering a sequence of historical stock prices, compared to MLP and CNN [12]. This finding proves the necessity of considering past stock prices for stock market analysis.

We adopt the RL algorithm to automatically manage assets in various portfolio allocation problems by performing optimal actions on a daily basis [1], [2]. However, when an agent takes an action in the current state using RL in [1] and [2], historical information cannot be considered due to the inherent Markov property of RL. Although the algorithms in [8] and [9] are proposed as a combination of LSTM and transformers with RL, only state is considered among MDP elements, and LSTM has a vanishing gradient problem.

The associate editor coordinating the review of this manuscript and approving it for publication was Huaqing Li^{ID}.

Hence, there exists a limit when the model learns a long sequence. Moreover, the transformer is not used as a decision model to predict actions.

We believe that RL would perform better if historical information could be considered as an ensemble model with transformers. Our study addresses this problem. In particular, we consider the RL model, which is able to utilize past stock information. To address this problem, the decision transformer [13] is a suitable model, which uses the GPT (Generative Pre-Training) model [14], where GPT has shown good performance in the field of natural language processing using the attention mechanism to predict the next token. By combining the GPT model with the RL technique, the GPT is able to predict the current action based on historical MDP elements through the attention mechanism. Thus, when modeling based on a decision transformer in stock investment, it can be trained using historical data.

The decision transformer updates the transformer using the mean squared error (MSE), which is a loss function. Using only MSE, it is difficult to exceed the performance of suboptimal trajectories prepared with training data. We prove this claim through an ablation study. Hence, in this paper, we propose TACR¹ (Transformer Actor-Critic with Regularization), a new RL algorithm, which applies the critic to the decision transformer to improve the overall performance of RL using past MDP elements. We are inspired by an actor-critic algorithm (Deep Deterministic Policy Gradient, DDPG) [15]. DDPG has been introduced to improve performance when the critic is added compared with that when only the actor is used. Additionally, we train the model offline using pre-generated suboptimal trajectories for imitating good actions and reducing learning time. we eliminate the need to collect additional data online. However, when only the critic is applied, the value function is inaccurate, leading to overestimation and reducing overall performance [16], since actor does not interact with the environment. To solve this problem, we apply the state-of-the-art regularization method [17] with an added behavior cloning to DDPG. The regularization method prevents actions from being overvalued by the critic.

TACR guarantees the highest returns compared to other RL and transformer algorithms [1], [2], [7], [18], [19], [20], [21] using various datasets from Dow Jones, US 50, HighTech, NDX, MDAX, and CSI. Additionally, we include the notion of the investment strategy turbulence in [1] and [22] for more stable returns.

Our main contributions are listed as follows:

- We propose Transformer Actor-Critic with Regularization (TACR), which predicts action with the GPT and then evaluates it using the critic to improve the performance of stock trading.
- We effectively train TACR offline based on generated suboptimal data. Furthermore, we add a regularization method to prevent overestimating the value of actions.

- We show the good performance of TACR, compared with the other state-of-the-art methods on stock trading using transformer and RL techniques.
- Through several experiments, we found that adding the critic network and regularization method improves performance, which generally leads to good results on various datasets.

II. RELATED WORK

We introduce two applications in the stock domain: stock prediction and stock trading. Several methods such as LSTM, transformer, and RL are proposed using various datasets.

A. LSTM

Since the stock datasets are easy to obtain, sufficient learning is possible using a deep learning algorithm. LSTM [23] can learn relatively longer sequences than RNN (Recurrent Neural Network) [12]. Hence, LSTM performs well in predicting the next stock price. It trains the model using time series data on historical stock prices [3]. An additional factor to consider when investing in stocks is news. Stock prices often depend on various news that affect the economy. References [4] and [5] show that adding news information or tweet features leads to better prediction performance compared with using only LSTM. This model uses VADER [24] to extract sentiment scores based on the news.

B. TRANSFORMER

Transformer [25] has had a significant impact on the field of natural language processing. Furthermore, Bert [26] and GPT [27] models, which focus on the encoder and the decoder, respectively, show better performance as the scale of the parameters increases. The transformer is also used in the stock domain to analyze historical data of stocks [6]. In addition, [7] considers the relationship between each stock and a market index. As each stock often depends on the market index, it helps to more accurately predict future stock prices.

C. REINFORCEMENT LEARNING (RL)

RL is suitable for the stock trading, where continuous decision making is required. Some representative RL algorithms (DDPG [15], TD3 [19], SAC [20], PPO [21], A2C [28]) are applied to stock trading in [1]. Another stock trading study [8] is conducted by combining A3C [28] with LSTM. When constructing the state features, RL shows the good performance by organizing the state into meaningful features related to the stock movement using SARL [2]. In fact, SARL improves the performance of RL by composing news embeddings as the features of the state. SARL feeds embedding into a hierarchical attention network [29] to train a binary classifier to predict the price movement.

D. TRANSFORMER + RL

To improve performance, the transformer and RL are used in a combined form. Reference [9] adds the transformer's attention mechanism to the RL model to improve the

¹Our code is available at: <https://github.com/VarML/TACR>

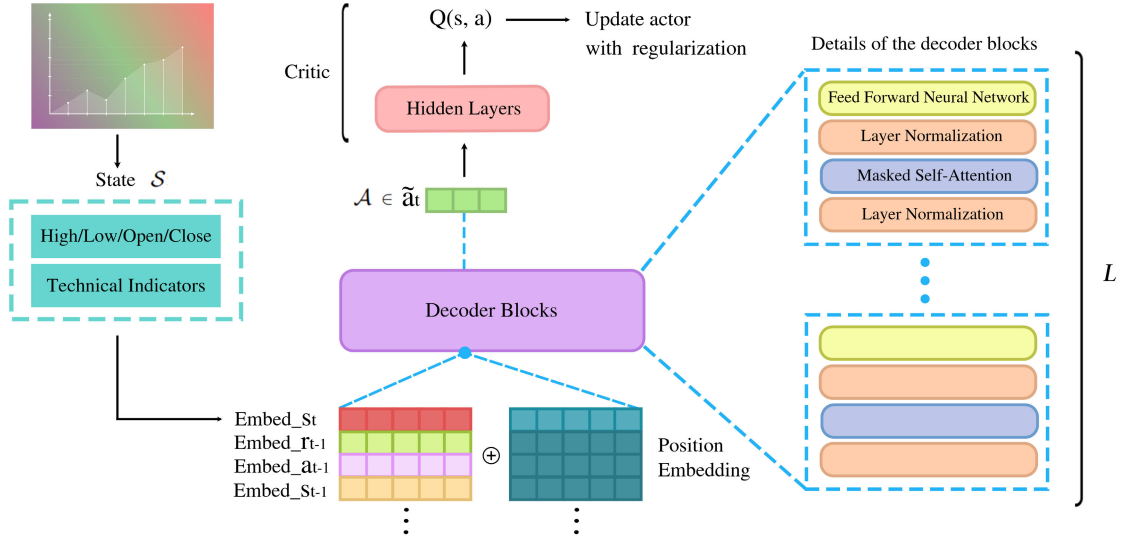


FIGURE 1. Framework of TACR. Various types of prices and technical indicators are composed into state features. The embeddings of past MDP elements and the current state are the inputs for the transformer. The action $\tilde{a}_t$ is predicted through several decoder blocks. Finally, the actor is updated via the approximated Q-function using the regularization method.

performance of stock trading. As mentioned in [7], the attention algorithm is also used to learn the interrelationships between stocks. Our TACR is also a combination of RL and transformer. However, TACR learns the correlation with MDP elements as input through offline RL algorithm. TACR is based on [13] and [30], which exhibits good performance in games and has not been used in the stock market domain. The advantage of TACR is that it can trade stocks using historical information via the transformer attention mechanism. Additionally, we provide the experiment results for various datasets to show the wide range of applicability.

III. BACKGROUND

In this Section, we construct the MDP for the stock trading problem, where we also explain how assets are managed in portfolio allocation. Then we introduce the notion of *investment strategy*, which is required to prevent the abrupt drop in portfolio profits. In fact, the investment strategy is able to compensate for the shortcomings of the learning model, leading to higher returns. In addition, we describe the transformer's attention mechanism to learn the relationship between MDP elements.

A. MARKOV DECISION PROCESS

We model the stock trading problem as a Markov decision process (MDP) [31]. The elements of MDP consisting of state, action, and reward are designed to model the portfolio allocation problem [32], [33], [34].

1) STATE SPACE $\mathcal{S}$

We construct the state space of MDP representing meaningful stock information such as opening/closing/high/low stock prices and technical indicators. Technical indicators include heuristic or pattern-based signals produced by price, volume,

and/or open interest of a security or contract used by traders following technical analysis.

2) ACTION SPACE $\mathcal{A}$

The action space is the set of the allocation weights $\mathbf{a}_t = \{a_{0,t}, a_{1,t}, \dots, a_{J,t}\}^\top$ satisfying $\sum_{j=0}^J a_{j,t} = 1$. Here, $a_{j,t}$ represents the size of allocation weight to invest in the j th stock for the current period t .

3) REWARD $\mathcal{R}$

The immediate reward is scalar and is defined as the sum of the rate of daily returns for each stock:

$$\boldsymbol{\rho}_t = \frac{\mathbf{p}_t \mathbf{C} \mathbf{1}}{\mathbf{p}_t} = \left(\frac{p_{0,t+1}}{p_{0,t}}, \frac{p_{1,t+1}}{p_{1,t}}, \dots, \frac{p_{J,t+1}}{p_{J,t}} \right)^\top \quad (1)$$

$$r_{t+1} = r(s_t, a_t) = \mathbf{a}_t \cdot (\boldsymbol{\rho}_t - \mathbf{1}), \quad (2)$$

where $\mathbf{p}_t \in \mathbb{R}_+^J$ denotes the closing price of stocks for the current period t . Note that (1) computes the ratio of the next day's stock prices to current stock prices. Ultimately, the agent maximizes the cumulative reward while increasing the action weights of the high-ratio stocks in (2).

B. INVESTMENT STRATEGY

Typically, RL algorithms [1], [35] perform well in the stock domain. However, in early 2020, the pandemic caused stock prices to drop sharply, leading to poor performance. To prevent this situation, an index that indicates the volatility of stock prices is required. When volatility is high, the action must be adjusted. We add an investment indicator to improve performance. This indicator is called *turbulence*, which was used in the NeurIPS Workshop 2020 [1] and was proposed by [22]:

$$\text{turbulence}_t = (\mathbf{p}_t - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{p}_t - \boldsymbol{\mu}) \in \mathbb{R}, \quad (3)$$

where $\mathbf{p}_t \in \mathbb{R}_+^J$ is the stock prices for the current period t , and $\boldsymbol{\mu} \in \mathbb{R}_+^J$ and $\boldsymbol{\Sigma} \in \mathbb{R}^{J \times J}$ are, respectively, the average and the covariance of past stock prices. This index acts as a constraint when taking actions. We describe in more detail in Subsection IV-A.

C. TRANSFORMERS

We describe how the attention mechanism is structured to compute the correlation for each input as follows:

$$\text{query} = \tilde{h}_l W_q, \text{key} = \tilde{h}_l W_k, \text{value} = \tilde{h}_l W_v \quad (4)$$

$$\tilde{h}_{l+1} = \text{Attention}(\text{query}, \text{key}, \text{value}) \quad (5)$$

$$= \text{Softmax}\left(\frac{\text{query} \cdot \text{key}}{\sqrt{d_k}}\right) \text{value}$$

$$\tilde{h}_{l+2} = \tilde{h}_{l+1} + \tilde{h}_l \quad (6)$$

$$\text{FFN}(\tilde{h}_{l+2}) = \max(0, \tilde{h}_{l+2} W_l + b_l) W_{l+1} + b_{l+1} \quad (7)$$

$$\tilde{h}_{l+3} = \tilde{h}_{l+2} + \text{FFN}(\tilde{h}_{l+2}), \quad (8)$$

where $\tilde{l}$ denotes the level of the hidden state in a specific layer. In (4), three weight matrices exist: query weights W_q , key weights W_k , and value weights W_v . The hidden state $\tilde{h}_l$ with layer normalization [36] applied is transformed into matrices query, key, and value by multiplying each weight. In (5), the similarities between each query and key are computed by using a dot product. To prevent the gradient from becoming almost zero in the Softmax function, $\sqrt{d_k}$, where d_k denotes the dimension of key, is divided for scaling. Furthermore, Softmax is applied to obtain the weights for the value. In (7), the FFN (Feed Forward Neural Network) comprises two linear layers, which use the ReLU activation function. In addition, the residual connection [37] in (6) and (8) alleviates the problem of gradient exploding or vanishing, which also allows faster learning by maintaining a stable gradient value.

IV. TRANSFORMER ACTOR-CRITIC WITH REGULARIZATION

Historical price information is an important factor in stock investments, since the past information can be used to more accurately analyze patterns of price flows. Previously, various investment strategies [38], [39] without machine learning techniques are constructed using past stock data to make high-yield investments. Recently, several approaches (e.g. [3], [7]) have been studied to consider correlation by utilizing past price information via machine learning algorithms such as LSTM and transformer.

In the case of RL, state information is usually insufficient when taking optimal actions, as RL is formulated based on the MDP framework. To solve this limitation and improve performance, we propose TACR (Transformer Actor-Critic with Regularization) using the decision transformer to utilize the previous stock information and MDP elements. In TACR, the decision transformer is considered as an actor predicting an action and then evaluates the action with a critic network adding regularization method to improve performance. As for

the specific structure of TACR, we use GPT [14] as the actor network and the Q-function as the critic network. The entire structure of TACR is depicted in Figure 1. We construct opening/closing/high/low stock prices and technical indicators (MACD, BOLL, RSI, CCI, DX, and SMA) as state features and then transform not only the current state but also previous states, actions, and rewards into embeddings of a specific dimension. TACR learns the correlation of each MDP element through the decoder blocks and predicts the current action as an output. Finally, the predicted action is evaluated by the critic, which is approximated through hidden layers, by which the parameters of the actor are updated. To explain TACR in more detail, we describe how the actor predicts the action in Subsection IV-A and how the actor updates the weights based on the critic with regularization in Subsection IV-B.

A. TRANSFORMER ACTOR

The mechanism of the transformer actor in TACR is to map the previous MDP elements to the current action. The actor network π consists of several decoder blocks that use the attention mechanism described in Subsection III-C and hidden layers as follows:

$$h_0 = MW_e + W_p \quad (9)$$

$$h_l = \text{Decoder_block}(h_{l-1}), \quad l = 1, \dots, L \quad (10)$$

$$\tilde{a} = \text{Softmax}(h_L W_L + b_L), \quad (11)$$

where $M = (r_{-u}, s_{-u}, a_{-u}, \dots, r_{-1}, s_{-1})$ is a matrix consisting of the token vectors of the previous MDP elements with length u indicating the sequence length of the previous tokens. In (9), the MDP elements are multiplied by the embedding matrix of weights W_e to represent the hidden state as an input and then added by position embedding weights W_p to include information of temporal continuity. Through L decoder blocks, the correlation of the embedding inputs is learned via (10). Finally, the action $\tilde{a} = \{\tilde{a}_0, \tilde{a}_1, \dots, \tilde{a}_J\}^\top$ with $\sum_{j=0}^J \tilde{a}_j = 1$, which is the allocation weight, is predicted through a linear transformation layer and a Softmax function to make the sum of the weights equal to 1 in (11).

We apply the turbulence strategy introduced in Subsection III-B as a constraint to prevent a sharp drop in the final step of predicting the action at each time step as follows:

$$\tilde{a}_t = \begin{cases} \mathbf{0}, & \text{if turbulence}_t > \text{threshold} \\ \pi(M), & \text{otherwise} \end{cases} \quad (12)$$

When the indices exceed a specific threshold, all shares held are sold. Otherwise, the agent is allowed to take action automatically. We apply this strategy to satisfy the same conditions only when compared to NeurIPS Workshop 2020.

B. BEHAVIOR CLONING AND CRITIC WITH REGULARIZATION

We train the model using an offline method rather than the off-policy method commonly used in stock trading. In other words, instead of interacting with the environment

and learning through trial and error, the agent imitates prepared suboptimal actions, expecting better performance and reducing learning time. In order to train the model in the offline method, suboptimal trajectories must be created by pairing the suboptimal actions corresponding to each state. Since historical data is abundant, we can easily create suboptimal trajectories as follows:

$$\text{score}_{j,t} = e^{\rho_{j,t} \cdot c} \quad (13)$$

$$\text{suboptimal_action}_{j,t} = \frac{e^{\text{score}_{j,t}}}{\sum_j e^{\text{score}_{j,t}}}, \quad (14)$$

where c is a constant in the range 1-5 and $\rho_{j,t}$ is introduced in (1). In (13), a score is given according to the rate of increase in the stock price, and in (14), a suboptimal action is defined to make the sum to 1 at each time step. The reward is computed as described in (2), and finally a set of states, actions, and rewards is completed at time t .

Although RL adopted in the stock field [1], [2] uses off-policy algorithms (not offline RL) to increase data efficiency using replay buffer, it has the disadvantage that it takes additional time to interact with the environment every episode to improve the policy. Thus, we train TACR offline using pre-generated suboptimal datasets similar to the case that the decision transformer uses expert trajectories of physics simulations provided in D4rl [40] for training. As a result, TACR can significantly reduce training time since TACR does not need to interact additionally with the environment and can improve its performance with relatively few iterations.

Additionally, in the offline RL algorithms, there is also a limitation of not interacting with the environment when updating a policy. In this case, agents tend to incorrectly estimate the value of actions for unseen states. The estimation failure problem leads to the poor performance as the optimal action is not taken. To evaluate more accurately out-of-distribution actions, the regularization method is required [17], which was modeled based on TD3 [19].

We change the way we update the policy using the critic for regularization as follows:

$$\begin{aligned} \pi &= \operatorname{argmax}_{\pi} \bar{J} \\ &= \operatorname{argmax}_{\pi} \mathbb{E}_{(s,a,r) \sim D} \left[\lambda Q(s, \pi(M)) - (\pi(M) - a)^2 \right], \end{aligned} \quad (15)$$

where π corresponds to the transformer actor and the sequence M indicates that a certain number of MDP elements is stacked. Intuitively, the behavior cloning regularization term is added to DDPG. This term causes the transformer actor to follow the distribution of actions in the suboptimal trajectories included in the dataset.

Note that λ is a hyperparameter that simultaneously controls maximizing Q and minimizing the behavior cloning term, which is highly sensitive to the scale of Q . In other words, the scalar λ denotes the strength of the regularizer.

Algorithm 1 TACR Algorithm

- 1: Randomly initialize critic weights θ^Q and transformer actor weights θ^π .
- 2: Initialize target weights $\theta^{Q'} \leftarrow \theta^Q, \theta^{\pi'} \leftarrow \theta^\pi$
- 3: Set the length of the sequence u and minibatch size n .
- 4: **for** each iteration **do**
- 5: **for** each environment step **do**
- 6: Sample a random minibatch of $n \times u$ transitions $(s_{i-u}, \dots, s_i, a_{i-u}, \dots, a_i, r_{i-u}, \dots, r_i, s_{i+1-u}, \dots, s_{i+1})$ from suboptimal trajectories.
- 7: $M_i = (r_{i-u}, s_{i-u}, a_{i-u}, \dots, r_i, s_i)$
- 8: Predict action $\tilde{a} = \pi(M_i | \theta^\pi)$ through transformer
- 9: Set $y_i = r + \gamma Q'(s_{i+1}, \pi'(M_{i+1} | \theta^{\pi'})) | \theta^{Q'}$
- 10: Apply mean squared error to update critic in (17):
 $\text{Loss} = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$,
 where $N = n \times u$.
- 11: Set hyperparameter α and $\lambda = \frac{\alpha}{\frac{1}{N} \sum_{(s_i, a_i)} |Q(s_i, a_i)|}$ in (16)
- 12: Update the transformer actor network in (18):
 $\nabla_{\theta^\pi} \bar{J} \approx \frac{1}{N} \sum_i \nabla_{\tilde{a}} \left\{ \lambda Q(s, \tilde{a} | \theta^Q) - (\tilde{a} - a)^2 \right\} |_{s=s_i, \tilde{a}=\pi(M_i)} \nabla_{\theta^\pi} \pi(M_i | \theta^\pi)$
- 13: Update the target weights:
 $\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$
 $\theta^{\pi'} \leftarrow \tau \theta^\pi + (1 - \tau) \theta^{\pi'}$
- 14: **end for**
- 15: **end for**

From data randomly sampled by N transitions (s_i, a_i) , λ is defined by

$$\lambda = \frac{\alpha}{\frac{1}{N} \sum_{(s_i, a_i)} |Q(s_i, a_i)|}, \quad (16)$$

where α is a hyperparameter that controls the sensitivity of λ . Note that (16) is considered to be normalization, where the denominator $\frac{1}{N} \sum_{(s_i, a_i)} |Q(s_i, a_i)|$ indicates the sample mean of $|Q(s_i, a_i)|$. In (16), since the Q is a non-differentiating scalar, it functions to normalize the learning rate by scaling the gradient $\nabla_a Q(s, a)$.

V. TACR ALGORITHM

We describe the proposed TACR algorithm in Algorithm 1. To trade many stocks simultaneously, we refer to DDPG, which is an actor-critic algorithm and can take continuous actions. DDPG performs well in various fields such as robotics, energy, and recommendation system [41], [42], [43]. Hence, we construct TACR based on DDPG.

The weights of critic and transformer actor are randomly initialized. Then the weights of the target networks are set. The hyperparameter u is a sequence length of the previous MDP elements to be used as inputs to the transformer actor. Then the $n \times u$ transitions are randomly sampled based on the minibatch size n and the sequence length u to break the correlation between data [44].

The transformer actor learns the correlation of stacked embedding MDP elements M through the attention network. The predicted action on the current state emerges as an output. To update the parameters of the networks, the critic is updated based on the parameter θ^Q using the mean squared error method given by (note that $Q(s_i, a_i|\theta^Q)$ denotes the approximation of Q)

$$\text{Loss} = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2. \quad (17)$$

Then the transformer actor parameter θ^π is updated based on the Q term and the behavior cloning term using the chain rule to $\bar{J}$, which is the objective function of DDPG using the regularization in (15) (note that $\pi(M_i^e|\theta^\pi)$ denotes the approximation of π):

$$\begin{aligned} \nabla_{\theta^\pi} \bar{J} \approx & \frac{1}{N} \sum_i \nabla_{\tilde{a}} \left\{ \lambda Q(s, \tilde{a}|\theta^Q) \right. \\ & \left. - (\tilde{a} - a)^2 \right\} \Big|_{s=s_i, \tilde{a}=\pi(M_i|\theta^\pi)} \nabla_{\theta^\pi} \pi(M_i|\theta^\pi). \end{aligned} \quad (18)$$

Finally, the weights of the target networks are slowly updated for learning stability: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$.

VI. EXPERIMENT

A. DATASETS

We use the same datasets as the state-of-the-art methods for comparison of TACR. In addition, to demonstrate the general applicability of TACR, we provide the experiment results using various datasets:

KDD 2021 [7] uses 50 US stocks, which we choose as the best performing dataset in KDD 21. Reference [7] splits the dataset into training data from 2007-01-03 to 2015-12-27 and testing data from 2016-01-04 to 2016-12-30.

AAAI 2020 [2] chooses nine stocks from the HighTech daily dataset [45]. Reference [2] splits the dataset into training data from 2006-10-20 to 2012-11-15 and testing data from 2012-11-16 to 2013-11-20.

NeurIPS Workshop 2020 [1] contains the DOW daily dataset comprising 30 prominent companies listed on stock exchanges in the US. Reference [1] splits the dataset into training data from 2009-01-01 to 2018-12-27 and testing data from 2019-01-01 to 2020-09-23.

NDX, MDAX, CSI are the stock indices of the US, Germany, and China, respectively. We select 30 random stocks for each index. We split the datasets into training data from 2009-01-01 to 2020-08-27 and testing data from 2020-09-01 to 2021-12-30.

B. PERFORMANCE

Transactions in all datasets are made on a daily basis. We adopt the most commonly used Portfolio value (PV) and Sharpe ratio (SR) [46] as metrics. PV is the final asset value,

and the SR evaluates the stability of the portfolio:

$$PV = pv_0 \prod_{t=1}^T (r_{t+1} - tf \sum_j |a_{j,t} - a_{j,t-1}| + 1) \quad (19)$$

$$SR = \frac{R_p - R_f}{\sigma_p}, \quad (20)$$

where pv_0 is \$1 million as initial amount, tf is 0.31% as transaction fee, and r_{t+1} is the sum of the rate of daily returns in (2). R_p and R_f denote, respectively, the average of the portfolio return and the risk-free rate, and σ_p denotes the standard deviation of the portfolio return. We set $R_f = 2\%$ as annual bank interest in (20).

We compare the performance of our model with other papers on portfolio allocation, and further include offline RL, off-policy RL, on-policy RL, and classic method as baselines:

- **KDD 21, AAAI 20, NeurIPS Workshop 20** [1], [2], [7]. These are the state-of-the-art papers showing good performance with portfolio allocation.
- **CQL**. Conservative Q-learning [18] is a representative algorithm of offline RL. We train CQL with suboptimal trajectories introduced in (14).
- **SAC**. Soft actor-critic [20] is an off-policy algorithm proposed to solve the limitation of application in the real world.
- **PPO**. Proximal policy optimization [21] is an on-policy RL that has stability and reliability and compensates for the flaw of policy gradient learning.
- **EW**. Equal weight is a naive baseline that assigns all stocks with the same weight in a way that does not use machine learning method.

We construct baselines with other papers, all model-free RL algorithms to show the superiority of TACR, and EW to show whether RL is practical for stock trading application. Figure 2 shows that TACR has the highest performance compared with various baselines. Moreover, we are able to confirm through the graphs that our model converges safely in Figure 3. Note that KDD 21 and AAAI 20 do not provide their codes and hyperparameters. Hence, we provide the comparison of our results with [7, Figure 1 (b)] and [2, Figure 3 (a)], respectively. Furthermore, Table 1 and 2 show the highest Sharpe ratio on most datasets. This means that our model produces stable returns compared to other algorithms. When compared to NeurIPS Workshop 20 and ours in Figure 2, the profits are not better in the beginning and middle. However, when looking at the Sharpe ratio in Table 2 computed for the entire period, ours is higher than NeurIPS Workshop 20. That means our profit is more stable based on the entire period. We compare AAAI 20 with the Sharpe ratio for the 6-month period shown in [2] in Table 3. The Sharpe ratio performance of KDD 21 can not be included in Table 1, since it does not provide details about the Sharpe ratio.

C. ABLATION STUDY

In TACR, critic and regularization method are applied in the decision transformer. We prove the necessity of each module

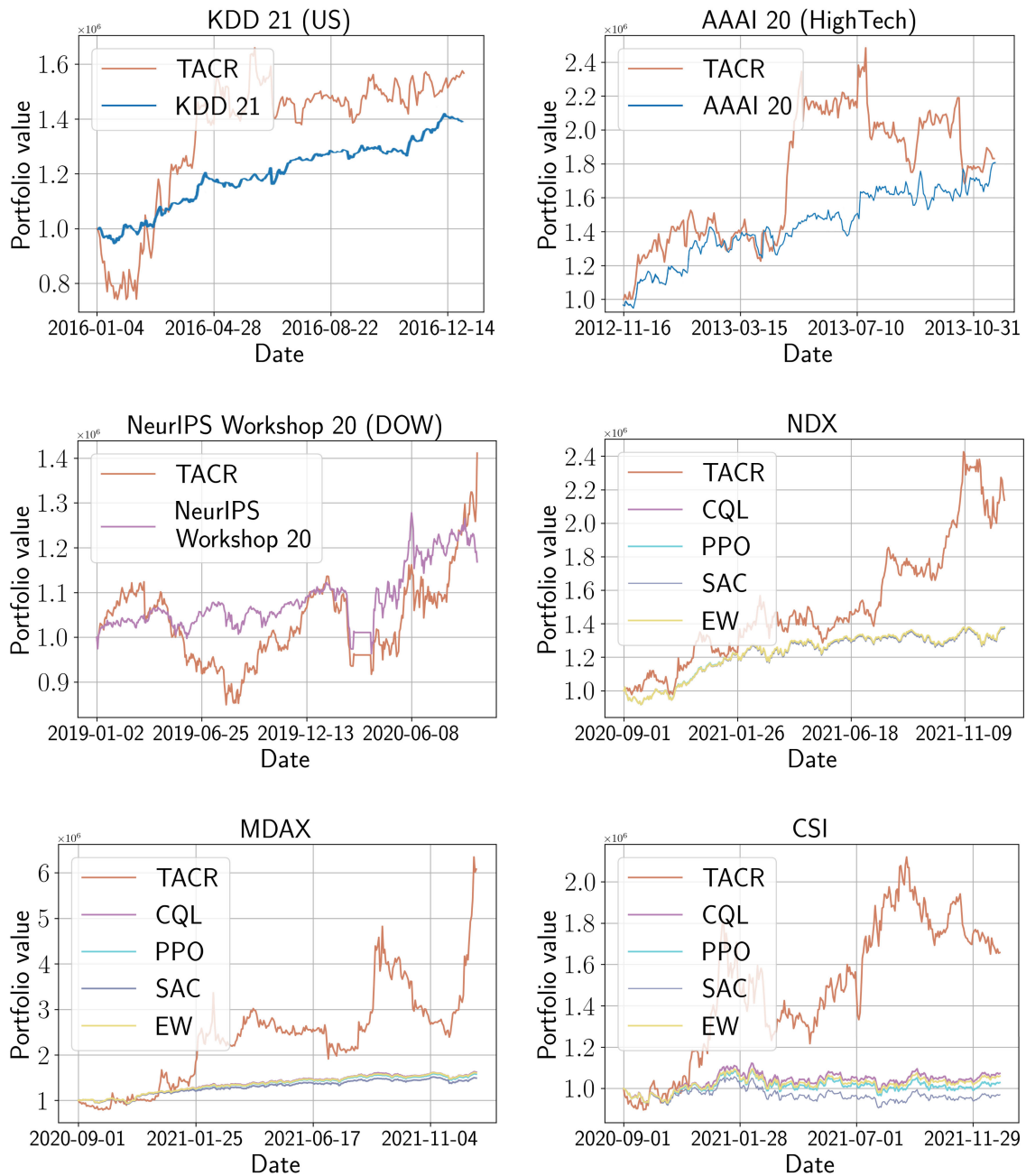


FIGURE 2. Comparison of the profit of TACR with other papers (KDD 21, AAAI 20, and NeurIPS Workshop 20), other existing RL approaches (CQL, SAC, and PPO), and naive baseline (EW) using six different datasets. TACR yields 13% (KDD 21), 1.1% (AAAI 20), 20.7% (NeurIPS Workshop 20), 54.8% (NDX), 274.8% (MDAX), and 54.5% (CSI) higher profits compared with other algorithms.

in TACR through an ablation study. In Table 3, our results show that the performance decreases on different datasets (NDX, MDAX, CSI) when each module is removed:

- TACR: Transformer Actor-Critic with Regularization;
- TACR w/o RG: TACR without the regularization (RG);
- TACR w/o RG, CT: TACR without the regularization and critic (CT).

Here, “w/o RG, CT” means that TACR only uses the decision transformer, which implies that TACR does not include

any module. We also show the performance of “TACR w/o RG”. As expected, the worst performance occurred with “w/o RG, CT”. By including the critic, “TACR w/o RG” shows the better performance than “w/o RG, CT”. By including the regularization method, we have TACR, which shows the better performance than “TACR w/o RG”. The regularization method prevents the overestimation of the value of actions and takes better actions on unseen observations. Ultimately, TACR, which considers

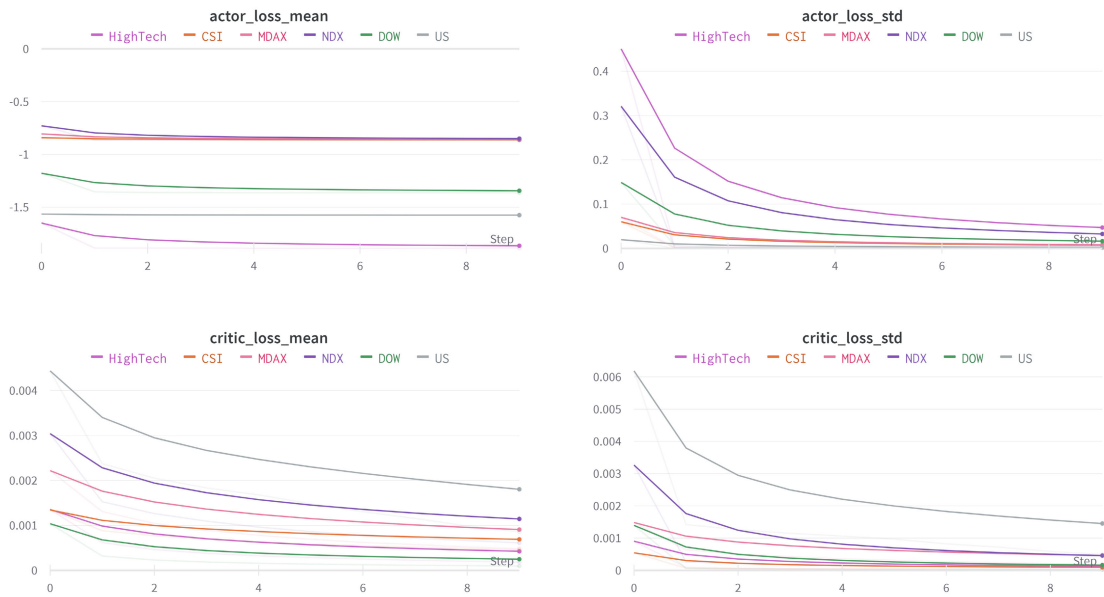


FIGURE 3. We train our model 10 iterations with 4000 steps per iteration. The graphs represent the training curves of the actor and critic for each dataset (HighTech, CSI, MDAX, NDX, DOW, US). It normally takes 30 minutes for training.

TABLE 1. Comparison of Sharpe ratios of TACR and other papers through each HighTech and DOW dataset.

model	TACR	AAAI 20	NeurIPS 20
HighTech	2.70	2.37	-
DOW	0.86	-	0.76

TABLE 2. Comparison of Sharpe ratios of TACR, other RL algorithms, and naive baseline with NDX, MDAX, and CSI.

model	TACR	CQL	PPO	SAC	EW
NDX	1.59	1.27	1.27	1.27	1.27
MDAX	1.81	1.96	1.85	1.61	1.94
CSI	1.00	0.22	0.36	0.31	0.27

TABLE 3. Ablation study.

	model	TACR	TACR w/o RG	TACR w/o RG, CT
NDX	PV	2.13	1.56	1.02
	SR	1.59	0.97	0.13
MDAX	PV	6.07	6.07	1.24
	SR	1.81	1.81	0.94
CSI	PV	1.65	0.85	0.27
	SR	1.00	-0.07	-3.20

all modules (critic and regularization), shows the best performance.

D. THE LENGTH OF THE SEQUENCE

The hyperparameter u can be used to adjust the length of the previous MDP sequence. Table 4 shows the change in

TABLE 4. Comparison of the sequence length.

	u (length)	20	10	1
NDX	PV	2.13	1.56	1.56
	SR	1.59	0.97	0.97
MDAX	PV	6.07	6.07	6.07
	SR	1.81	1.81	1.81
CSI	PV	1.65	1.38	1.03
	SR	1.00	0.74	0.24

performance with increasing u on various datasets (NDX, MDAX, CSI). We compare the performance along the length of the sequence in the range (1, 10, 20). Most datasets show good results when increasing u . Simultaneously, profit and stability increase. This means that the longer the past MDP elements are considered, the closer to optimal action is taken. It is presumed that the price flow factor influences the action taken. TACR complements the shortcomings of RL based on insufficient observation and learns the relationships of past information well through the attention network.

VII. CONCLUSION

In stock trading, it is essential to analyze the market using historical data. We propose Transformer Actor-Critic with Regularization (TACR), which includes the critic network to the decision transformer and the regularization method. In addition, we train our model efficiently using suboptimal trajectories offline. TACR enables RL method to consider not only past stock data but also previous MDP elements when taking optimal actions, which is different from standard RL techniques formulated based on the MDP framework.

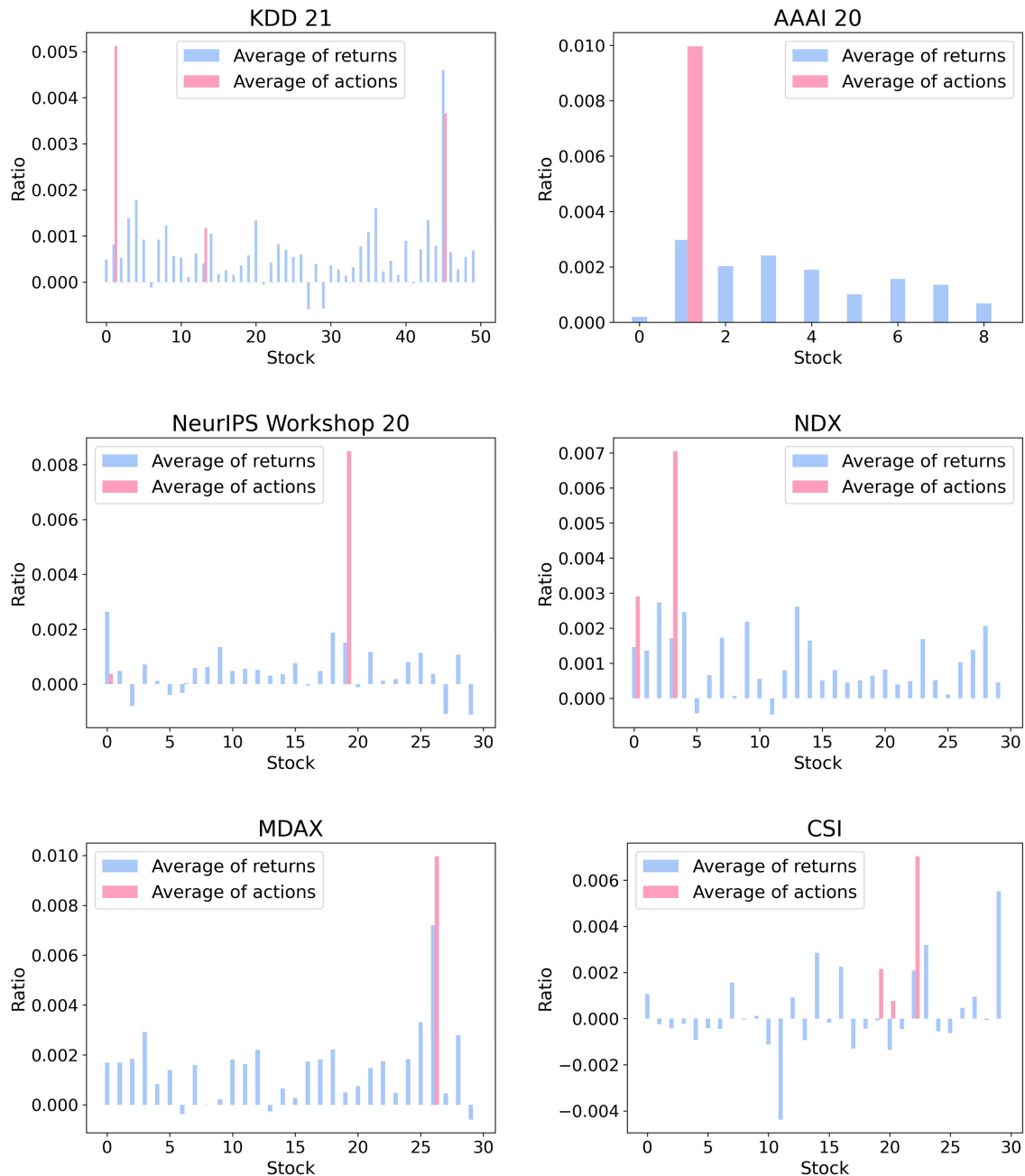


FIGURE 4. The x-axis represents each stock, and the y-axis represents the average of actions and the average of returns. The average of the actions is normalized by dividing by 100 for ease of viewing relative to the average of the returns.

Furthermore, TACR prevents the overestimation of the values of the actions. Compared with state-of-the-art methods, TACR shows good performance for various datasets. Similar to other methods, however, when stock prices fall sharply due to pandemic, TACR also shows a sharp drop if TACR does not use the investment strategy. In this case, we may assume that some external factors such as pandemics affect the stock price patterns, which lead to the out of the distribution of the environment when testing data. For the future work, we would like to study the model, which can analyze external factors

under unseen observations. For example, by considering news information embeddings that can affect stock prices and using it as an additional input other than MDP to the transformer, the model would be able to learn external factors as well.

APPENDIX A EXPERIMENTAL DETAILS

A. SOFTWARE

- Linux Ubuntu 20.04.3
- Python 3.9

TABLE 5. Transformer and critic hyperparameters of TACR for US 50, HighTech, DOW, NDX, MDAX, and CSI dataests.

Hyperparameter	Value
Number of layers	5
Number of attention heads	1
Embedding dimension	128
Batch size	64
Context length	20 (NDX, MDAX, CSI), 40 (US, HighTech, DOW)
Activation function	ReLu
Dropout	0.1
Learning rate	10^{-4}
Grad norm clip	0.1
Weight decay	10^{-4}
Learning rate decay	Linear warmup for first 10^5 training step
Critic hidden dim	(512 x 316), (316 x 1)
Critic hidden layers	2
Critic activation function	ReLu
Critic learning rate	10^{-4} (HighTech) 10^{-6} (Others)
seed	0
α	0.1 - 2.0

- Pytorch 1.10.1
- Transformers 4.5.1
- Gym 0.19.0
- Anaconda 4.12.0
- Numpy 1.21.2

B. HARDWARE

- Intel Core i9-10980XE CPU at 3.00GHz.
- GPU RTX A5000

C. HYPERPARAMETERS

We set the hyperparameters of the transformer model GPT as default values. The hyperparameters that we focus on are context length u and α , where α is defined in (16). While fixing other hyperparameters by the specific values, we show the best performance on all datasets by tuning only the hyperparameters u and α . Detailed hyperparameters are shown in Table 5. We experiment by increasing the hyperparameter u from 1 to 60 with increments of 10. Performances including Sharpe ratio and profit shown in Table 3 increase as the context length u increases. However, when the length u is over 40, there is no further improvement. Furthermore, we experiment by increasing α from 0.1 to 2.0 with increments of 0.1. When α value is 1.6 in the US, 1.4 in the DOW, 2.0 in the HighTech, 0.9 in the NDX, MDAX, and CSI, the highest Sharpe ratio and profit are achieved, and MDAX is not sensitive to the value of α . US 50, HighTech, and DOW datasets are used in [1], [2], [7], and [45], respectively. Note that in our paper, we reproduce the NeurIPS Workshop 2020 from <https://github.com/AI4Finance-Foundation/FinRL>. However, we are not able to reproduce the results of [2] and [7], as these papers do not provide its code and hyperparameters. To model

TABLE 6. State features for additional experiments.

	Features 1	Features 2	w/o TI
State	Opening	Opening	Opening
	Closing	Closing	Closing
	High	High	High
	Low	Low	Low
	MACD	TRIX_30	
	BOLL_UB	MVAR_30	
	BOLL_LB	EMA_30	
	RSI_30	RSI_20	
	CCI_30	WR_30	
	DX_30	ATR	
	SMA_30, 60	DMA	

our algorithm, we refer to the transformer and regularization technique as below:

- <https://github.com/kzl/decision-transformer>
- https://github.com/sfujim/TD3_BC

D. ENVIRONMENT

We use NeurIPS Workshop 2020's code for a stock trading environment, stock data, and allocation experiments using RL. We construct the state including the technical indicators (MACD, BOLL, RSI, CCI, DX, and SMA) and prices (high, low, opening, and closing) for each stock. The HighTech dataset considers an 108-dimensional state. Except for HighTech, the other datasets use a 360-dimensional state. The dimension of action is determined by the number of stocks invested in. Additionally, the turbulence threshold is set to 100 as an investment strategy. We use the indicator to make fair comparisons between ours and NeurIPS Workshop 20, since NeurIPS Workshop 20 uses the indicator to prevent a sharp drop. We set the indicator threshold to 100 and then take no action when the indicator exceeds the threshold. The investment strategy is used only for the NeurIPS Workshop 2020 dataset [1], and the other datasets show good performance without the investment strategy.

APPENDIX B ADDITIONAL EXPERIMENTS

Other RLs (CQL, PPO, SAC) tend to assign almost equal weights when taking actions. Hence, their performances are similar to the EW baseline. As a first additional experiment, we show a graph of how TACR can outperform other algorithms by assigning actions.

The construction of meaningful state information helps the agent to take high-reward actions. Hence, as shown in Figure 1, we add technical indicators to the state feature as well as the opening/closing/high/low stock prices to take high-reward actions. To show the impact of technical indicators, we provide the second experimental results of the HighTech, DOW, NDX, MDAX, and CSI datasets for the case with different sets of technical indicators and without the technical indicators (see Table 6). KDD 21 dataset is not included since it does not provide technical indicators.

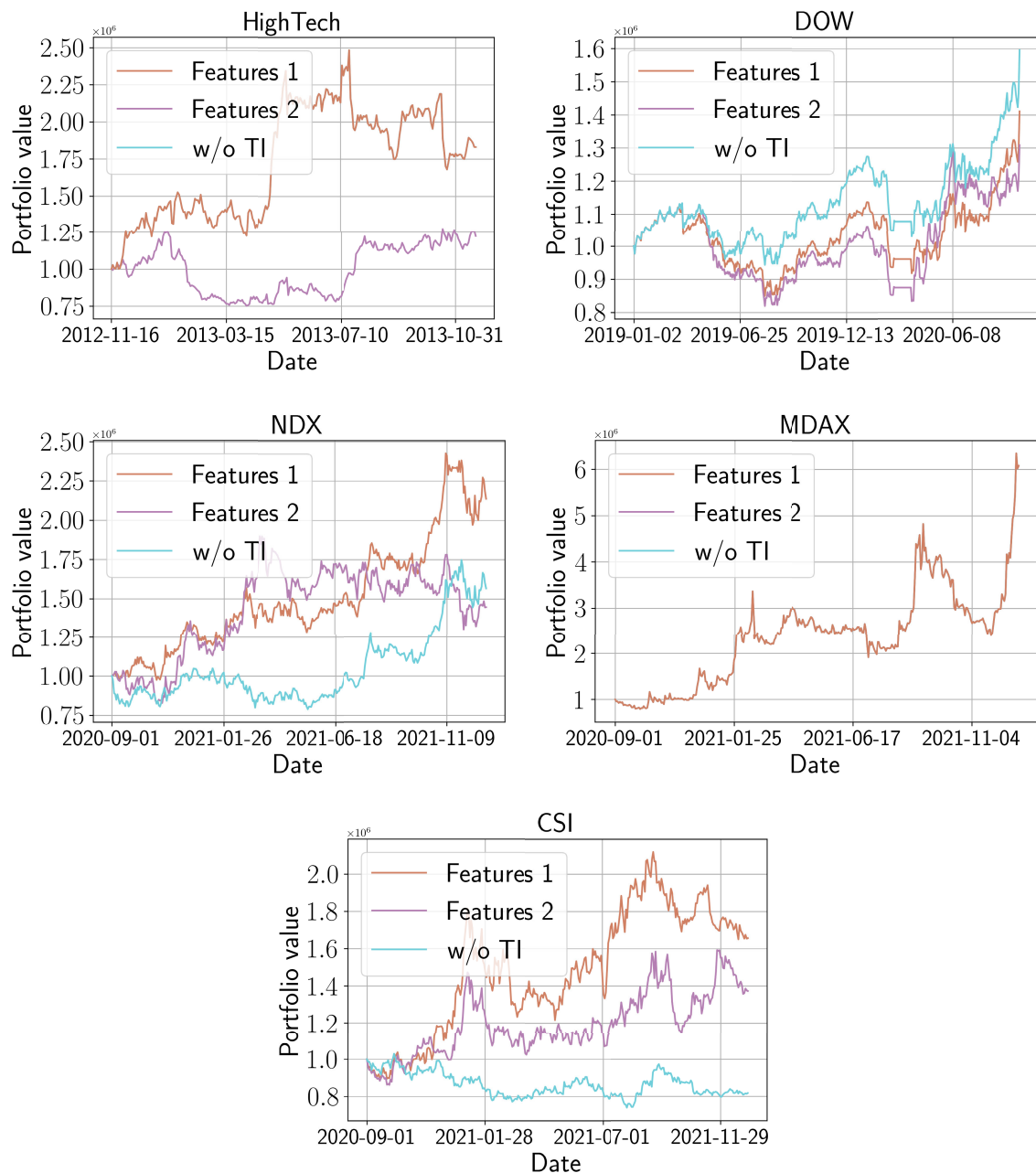


FIGURE 5. Comparison of different sets of technical indicators for the HighTech, DOW, NDX, MDAX, and CSI datasets (see Features 1, Features 2, w/o TI in Table 6).

We provide the technical indicators used in the main experiments and additional experiments and divide them by type as follows:

- **Moving average:**

- Moving Average Convergence Divergence (MACD)
- Exponential Moving Average (EMA)
- Different of Moving Average (DMA)
- Directional Movement Index (DX)
- Triple Exponential Moving Average (TRIX)
- Simple Moving Average (SMA)

- **Overbought/Oversold:**

- Relative Strength Index (RSI)
- Williams Overbought/Oversold index (WR)
- Commodity Channel Index (CCI)

- **Volatility:**

- Moving Variance (MVAR)
- Average True Range (ATR)
- Bollinger (BOLL).

We divide technical indicators into three categories. The moving average is an indicator that identifies the trend of

TABLE 7. Comparison of technical indicators.

		Features 1	Features 2	w/o TI
Hightech	PV	1.83	1.22	1.83
	SR	1.35	0.65	1.35
DOW	PV	1.41	1.30	1.59
	SR	0.86	0.65	1.15
NDX	PV	2.13	1.44	1.56
	SR	1.59	0.77	0.97
MDAX	PV	6.07	6.07	6.07
	SR	1.81	1.81	1.81
CSI	PV	1.65	1.37	0.81
	SR	1.00	0.72	-0.43

stock prices. The overbought/oversold indicator detects large changes in stock quantity. The volatility indicator shows the stocks' stability by detecting stock price changes. Note that the detailed information on technical indicators can be found in <https://www.investopedia.com>.

A. RESULTS

As a result of the first additional experiment, Figure 4 shows the average of the returns and actions assigned to each stock over the entire trading period. Looking at each graph, we can see that TACR allocates the majority to a few specific stocks rather than allocating them evenly across multiple stocks. Except for the CSI dataset, most datasets assign most actions to high average returns. Note that we do not provide information about cumulative returns or average returns when training the model, however, TACR finds stocks with good growth rates.

A detailed list of Features 1, Features 2, and without the technical indicators (w/o TI) is shown in Table 6. We compute each technical indicator with respect to the closing price. In Table 6, “_20, 30, 60” means that the technical indicators are computed every 20, 30, and 60 days, respectively. “_LB, UB” indicates lower bound and upper bound, respectively. Note that Features 1 is used in our main experiment in Section “Experiment”, whereas Features 2 includes additional technical indicators for additional experiments.

For the second additional experiment, in Figure 5 and Table 7, when training a model with Features 1, most datasets show good performance. Features 1 is meaningful information that an agent can identify patterns in stock prices. However, in the DOW dataset, not including any technical indicators shows the highest performance, and training a model with Features 2 gives poor results in most datasets. Some technical indicator information interferes with the analysis of the pattern of stock prices. This implies that the choice of technical indicators is crucial to improve the performance.

USE OF AI TOOLS DECLARATION

The authors declare they have not used Artificial Intelligence (AI) tools in the creation of this article.

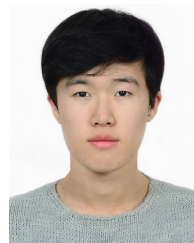
CONFLICT OF INTEREST

The authors declare no conflict of interest in this paper.

REFERENCES

- [1] X.-Y. Liu, H. Yang, Q. Chen, R. Zhang, L. Yang, B. Xiao, and C. D. Wang, “FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance,” 2020, *arXiv:2011.09607*.
- [2] Y. Ye, H. Pei, B. Wang, P.-Y. Chen, Y. Zhu, J. Xiao, and B. Li, “Reinforcement-learning based portfolio management with augmented asset movement prediction states,” in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 1, pp. 1112–1119.
- [3] D. M. Q. Nelson, A. C. M. Pereira, and R. A. de Oliveira, “Stock market’s price movement prediction with LSTM neural networks,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 1419–1426.
- [4] X. Li, Y. Li, H. Yang, L. Yang, and X.-Y. Liu, “DP-LSTM: Differential privacy-inspired LSTM for stock prediction using financial news,” 2019, *arXiv:1912.10806*.
- [5] D. Chen, Y. Zou, K. Harimoto, R. Bao, X. Ren, and X. Sun, “Incorporating fine-grained events in stock movement prediction,” 2019, *arXiv:1910.05078*.
- [6] Q. Ding, S. Wu, H. Sun, J. Guo, and J. Guo, “Hierarchical multi-scale Gaussian transformer for stock movement prediction,” in *Proc. IJCAI*, 2020, pp. 4640–4646.
- [7] J. Yoo, Y. Soun, Y.-C. Park, and U. Kang, “Accurate multivariate stock movement prediction via data-axis transformer with multi-level contexts,” in *Proc. 27th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2021, pp. 2037–2045.
- [8] E. S. Ponomarev, I. V. Oseledets, and A. S. Cichocki, “Using reinforcement learning in the algorithmic trading problem,” *J. Commun. Technol. Electron.*, vol. 64, no. 12, pp. 1450–1457, Dec. 2019.
- [9] J. Wang, Y. Zhang, K. Tang, J. Wu, and Z. Xiong, “AlphaStock: A buying-winners-and-selling-losers investment strategy using interpretable deep reinforcement attention networks,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Jul. 2019, pp. 1900–1908.
- [10] B. Li and S. C. H. Hoi, “On-line portfolio selection with moving average reversion,” 2012, *arXiv:1206.4626*.
- [11] L. Gao and W. Zhang, “Weighted moving average passive aggressive algorithm for online portfolio selection,” in *Proc. 5th Int. Conf. Intell. Human-Machine Syst. Cybern.*, vol. 1, Aug. 2013, pp. 327–330.
- [12] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [13] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, “Decision transformer: Reinforcement learning via sequence modeling,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 15084–15097.
- [14] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [15] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” 2015, *arXiv:1509.02971*.
- [16] S. Levine, A. Kumar, G. Tucker, and J. Fu, “Offline reinforcement learning: Tutorial, review, and perspectives on open problems,” 2020, *arXiv:2005.01643*.
- [17] S. Fujimoto and S. S. Gu, “A minimalist approach to offline reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 20132–20145.
- [18] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-learning for offline reinforcement learning,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1179–1191.
- [19] S. Dankwa and W. Zheng, “Twin-delayed DDPG: A deep reinforcement learning technique to model a continuous movement of an intelligent robot agent,” in *Proc. 3rd Int. Conf. Vis., Image Signal Process.*, Aug. 2019, pp. 1–5.
- [20] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017, *arXiv:1707.06347*.
- [22] M. Kritzman and Y. Li, “Skulls, financial turbulence, and risk management,” *Financial Analysts J.*, vol. 66, no. 5, pp. 30–41, Sep. 2010.

- [23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [24] C. Hutto and E. Gilbert, "VADER: A parsimonious rule-based model for sentiment analysis of social media text," in *Proc. Int. AAAI Conf. Web Social Media*, 2014, vol. 8, no. 1, pp. 216–225.
- [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 1–11.
- [26] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," 2018, *arXiv:1810.04805*.
- [27] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving language understanding by generative pre-training," Open AI, Tech. Rep., 2018. [Online]. Available: https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/language-unsupervised/language_understanding_paper.pdf
- [28] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1928–1937.
- [29] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics, Human Lang. Technol.*, 2016, pp. 1480–1489.
- [30] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 1273–1286.
- [31] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [32] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," 2017, *arXiv:1706.10059*.
- [33] Z. Liang, H. Chen, J. Zhu, K. Jiang, and Y. Li, "Adversarial deep reinforcement learning in portfolio management," 2018, *arXiv:1808.09940*.
- [34] X. Li, Y. Li, Y. Zhan, and X.-Y. Liu, "Optimistic bull or pessimistic bear: Adaptive deep reinforcement learning for stock portfolio allocation," 2019, *arXiv:1907.01503*.
- [35] X.-Y. Liu, Z. Xiong, S. Zhong, H. Yang, and A. Walid, "Practical deep reinforcement learning approach for stock trading," 2018, *arXiv:1811.07522*.
- [36] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
- [37] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [38] L. C. MacLean, E. O. Thorp, and W. T. Ziemba, *The Kelly Capital Growth Investment Criterion: Theory and Practice*, vol. 3. Singapore: World Scientific, 2011.
- [39] A. Ang, "Mean-variance investing," Columbia Bus. School, Res. Paper 12/49, Aug. 2012.
- [40] J. Fu, A. Kumar, O. Nachum, G. Tucker, and S. Levine, "D4RL: Datasets for deep data-driven reinforcement learning," 2020, *arXiv:2004.07219*.
- [41] X. Zhao, L. Zhang, L. Xia, Z. Ding, D. Yin, and J. Tang, "Deep reinforcement learning for list-wise recommendations," 2017, *arXiv:1801.00209*.
- [42] C. Sampedro, A. Rodriguez-Ramos, H. Bayle, A. Carrio, P. de la Puente, and P. Campoy, "A fully-autonomous aerial robot for search and rescue applications in indoor environments using learning-based techniques," *J. Intell. Robot. Syst.*, vol. 95, no. 2, pp. 601–627, Aug. 2019.
- [43] C. Qiu, Y. Hu, Y. Chen, and B. Zeng, "Deep deterministic policy gradient (DDPG)-based energy harvesting wireless communications," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8577–8588, Oct. 2019.
- [44] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, and G. Ostrovski, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [45] X. Ding, Y. Zhang, T. Liu, and J. Duan, "Using structured events to predict stock price movement: An empirical investigation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1415–1425.
- [46] A. W. Lo, "The statistics of Sharpe ratios," *Financial Analysts J.*, vol. 58, no. 4, pp. 36–52, Jul. 2002.



NAMYONG LEE received the B.S. degree in multimedia from Chonnam National University, Chonnam, South Korea, in 2021, and the M.S. degree in artificial intelligence from Hanyang University, Seoul, South Korea, in 2023. His research interests include the development of finance and robotics using reinforcement learning and computer vision.



JUN MOON (Senior Member, IEEE) received the B.S. degree in electrical and computer engineering and the M.S. degree in electrical engineering from Hanyang University, Seoul, South Korea, in 2006 and 2008, respectively, and the Ph.D. degree in electrical and computer engineering from the University of Illinois at Urbana–Champaign, USA, in 2015. From 2008 to 2011, he was a Researcher with the Agency for Defense Development, South Korea. From 2016 to 2019, he was an Assistant Professor with the School of Electrical and Computer Engineering, Ulsan National Institute of Science and Technology, South Korea. From 2019 to 2020, he was an Associate Professor with the School of Electrical and Computer Engineering, University of Seoul, South Korea. He is currently an Associate Professor with the Department of Electrical Engineering, Hanyang University. His research interests include stochastic optimal control and filtering, networks, reinforcement learning, distributed control, networked control systems, and mean field games. He was a recipient of the Fulbright Graduate Study Award, in 2011.

...